

University Paris Saclay

Master M1 Data Science

Key-Based SHACL Shape Generation and Validation for RDF Datasets with Property Mapping Support

SALMA ALKHALILY

*S*upervisor:

Professor Fatiha Sais

1 January 2025 - 1 April 2025

Abstract

Generating SHACL shapes is important for validating that the data graphs satisfy a set of conditions. SHACL shapes can also be viewed as a description of the data graphs that meet these conditions. Such descriptions have various uses beyond validation, including the creation of user interfaces, the generation of code, and data integration. The validation process produces a report that contains violations and describes the conditions that were violated. This project develops a tool to generate keys for RDF data sources and uses these keys to create SHACL shapes for validation purposes. The generated SHACL shapes are validated against the original RDF data source, other RDF data sources that share the same keys, or those that share the same keys but have different property names. Validation reports are produced as a result of the validation process, providing detailed explanations of any violations, the total number of violations, a breakdown for each breached condition, and information on which data instances and keys have caused the breaches.

In this project, two scenarios are considered. In the first scenario, SHACL shapes are generated directly from the keys and validated against RDF data sources that share the same property structures, enabling an evaluation of the keys' effectiveness across these RDF data sources. In the second scenario, translated properties corresponding to those used in generating keys are used to generate SHACL shapes, enabling validation across RDF data sources that share the same structure but have different property names. This approach ensures compatibility across multiple data sources with varying naming conventions. To support these scenarios, two SHACL constraints are defined to detect and handle duplicates. The Uniqueness Constraint verifies that each key is unique and identifies duplicates, while the sameAs Rule ensures that detected duplicates are properly linked using owl:sameAs.

The workflows were implemented using Apache Jena, a robust semantic data processing framework, and a user-friendly interface was developed with JavaFX and minimal CSS. This report provides a detailed explanation of each scenario, their methodologies, and results, supported by examples and illustrations.

Contents

1 Introduction **1**

1.1 Background 1

1.2 Problem Statement 2

1.3 Objectives of the Study 3

1.4 Methodology 3

1.5 Structure of the Report 4

2 Approach **5**

2.1 Conversion to N-Triples Format 5

2.2 Generating Keys Process 7

2.2.1 Overview 7

2.2.2 Challenges in Key Discovery 7

2.3 SAKey Tool for Key Discovery 7

2.3.1 Overview 7

2.3.2 SAKey Limitation 8

2.3.3 SAKey Parameters 8

2.3.4 Running SAKey 8

2.3.5 Generating keys process 9

2.4 Generating SHACL Shapes 10

2.4.1 Overview 10

2.4.2 Generating SHACL Shapes Based on Keys 11

2.4.3 Generating SHACL Shapes Based on Keys and Translated Properties 12

2.4.4 Uniqueness Constraint 14

2.4.5 SameAs Rule 15

2.4.6 Defining the Target Nodes 17

2.5 Validation Process 18

2.5.1 Overview: 18
2.5.2 Validation Steps: 18
2.5.3 Data Processing Algorithm: 19
2.5.4 Validation Report 19
2.6 Conclusion 21

List of Figures **22**

List of Tables **24**

Chapter 1

Introduction

1.1 Background

The Resource Description Framework (RDF)[1] is a standard model for data interchange on the Web. It is designed to facilitate data merging, even when the underlying schemas differ, and to support the evolution of schemas over time without requiring changes from all data consumers. RDF uses Uniform Resource Identifiers (URIs) to name relationships between resources, forming "triples" (subject-predicate-object). This simple and flexible model enables structured and semi-structured data to be mixed, exposed, and shared across different applications.

RDF graphs are essentially directed, labeled graphs where edges represent the named links between resources (represented as graph nodes). This graph structure provides a model for RDF and is commonly used in visual explanations of the framework.

SHACL (Shapes Constraint Language)[2] is a powerful language used to validate RDF graphs against a set of conditions. These conditions are specified in the form of shapes that are also RDF graphs.

The validation process involves checking a data graph (the RDF data that are being validated) against a shapes graph (the set of conditions defined in SHACL). The results of the validation indicate whether the data graph conforms to the shapes and detect violations.

As RDF datasets grow in size and complexity, especially when integrating data from multiple sources, ensuring their consistency becomes increasingly challenging. SHACL is important for enforcing data integrity by validating RDF data sources. However, manually

[1] https://www.w3.org/RDF/

[2] https://www.w3.org/TR/shacl/

generating SHACL shapes for large data sets is a difficult and great challenge.

1.2 Problem Statement

With the growing volume of RDF data, it is essential to validate data across multiple sources, especially when those sources share similar structures but may use different property names. For example, consider a dataset that contains books and their authors, along with another dataset that includes newly published books by the same authors. Although both datasets share the same structure, they may use different property names; for instance, one dataset might use the *title* property, while another uses *book-title*.

In this case, it is inefficient to generate keys separately for each dataset before creating SHACL shapes and validating them. Instead, keys should be generated once from the first dataset. For the second dataset, SHACL shapes can be generated based on these predefined keys and the translated property names. This approach ensures that the keys remain valid across datasets with similar structures and avoids redundant processing.

However, generating SHACL shapes based on these keys—or based on these keys along with mapped property names, ensuring consistent validation across RDF sources and generating validation reports with details about violations can be a complex task.

Moreover, in cases where a third RDF dataset has the exact same structure and property names as the first, regenerating SHACL shapes again is not only redundant but also conceptually incorrect. Since the keys and shapes are already defined and valid for that structure, generating them again is a duplicated effort. Therefore, when datasets are structurally identical, SHACL shapes should not be regenerated; instead, existing shapes should be reused to maintain consistency in validation.

This project addresses these challenges by providing a tool that generates SHACL shapes from keys extracted from RDF datasets or from keys and translated properties. The tool validates these shapes against both the original and other structurally similar datasets, generates validation reports detailing exceptions or constraint violations, and summarizes the total number of detected issues.

1.3 Objectives of the Study

The main objectives of this project are to develop a tool for:

- Generating keys derived from the properties of an RDF data source using an external tool.
- Comparison of two scenarios of SHACL shape generation:
  - Generating SHACL shapes from keys derived from the properties of an RDF data source using an external tool.
  - Generating SHACL shapes for equivalent datasets based on pre-generated keys and translated property names.
- Identifying which keys are invalid across equivalent RDF data sources.
- Converting any RDF data format to N-Triples before the key generation and validation process.
- Validating SHACL shapes against an RDF data source and generating a validation report that provides a detailed explanation of the breached constraints by specific data instances.

1.4 Methodology

The proposed tool was implemented using Apache Jena[1] , a widely used semantic web framework that provides RDF processing and SHACL validation capabilities. Jena was chosen due to its efficiency in handling large RDF datasets and its support for SHACL constraint validation.

Additionally, a user-friendly interface was developed using JavaFX[2] with minimal CSS[3] styling to enhance usability. The interface allows users to input RDF data sources, generate SHACL shapes, apply validation, visualize validation reports, and save the results of each operation to a file..

The tool follows a structured workflow:

1. Convert RDF data format to N-triples.

[1]https://jena.apache.org/
[2]https://openjfx.io/
[3]https://devdocs.io/css/

2. Extract keys from RDF data sources using an external tool.

3. Generate SHACL shapes based on the extracted keys or based on both keys and translated property names.

4. Validate the generated SHACL shapes against the RDF data sources.

5. Produce a validation report detailing conformance results and violations.

1.5 Structure of the Report

This report is structured as follows:

- Chapter 2 explains the approach for converting RDF data format to N-triples, generating keys, creating SHACL shapes for RDF data sources, and validating the generated shapes against RDF data sources, including the two predefined scenarios.
- Chapter 3 shows different experiments with its results and discuss the results.
- Chapter 4 concludes the report and suggests areas for future work.
- Appendix A contains a full example follows the approach with user interfaces.

Chapter 2

Approach

This chapter describes the approach for generating SHACL shapes and validating them against RDF data sources. The implementation of this approach consists of four main parts:

- Converting RDF data format to N-Triples format.
- Generating keys.
- Generating SHACL shapes.
- Validating SHACL shapes against RDF data.

Each part is described in detail in the following sections.

2.1 Conversion to N-Triples Format

The first step in the implementation process is converting RDF data into the N-Triples[1] (NT) format. This is necessary because the SAKey tool used for key discovery accepts only RDF datasets in NT format. Since RDF data can be represented in various serialization formats, a conversion mechanism was implemented to support multiple input types.

The following RDF serialization formats are supported for conversion:

- **N-Quads (NQ)** [2] **:** N-Quads is a line-based, plain text format for encoding an RDF dataset.

[1] https://www.w3.org/TR/n-triples/
[2] https://www.w3.org/TR/n-quads/

- **Turtle (TTL):** A compact and human-readable syntax for RDF that uses prefixes and shorthand notations.
- **RDF/XML/RDFXML:** The original RDF syntax that serializes RDF as an XML document, widely supported but less human-friendly, and typically the three are differentiated by file naming.

Algorithm 1: RDF File Conversion Process

Input: Input file path, Output file path
Output: Converted RDF file in NT format
Determine the input file format based on its file extension
if input file format is unsupported **then**
    Abort the process with a message
end if
Check whether the input format supports quads
if quads are involved **then**
    Load the input data into a Dataset
    if the output format does not support quads **then**
        Merge all named graphs into the default graph
        Write the merged model to the output file
    else
        Write the dataset directly to the output file
    end if
else
    Load the input data into a Model
    Write the model to the output file
end if

Quads (N-Quads) have a 4th element: a graph name. Therefore, we need a Dataset (collection of multiple graphs) instead of a single Model, and merge all named graphs into the default graph because triple-based formats (like N-Triples) do not support graph names. Triples (N-Triples, Turtle, RDF/XML) involve only subject, predicate, object, so a simple Model is enough.

2.2 Generating Keys Process

2.2.1 Overview

A key is a set of properties whose values uniquely identify every resource of a dataset [?]. Keys can be used as logical rules to clean or link data when high precision is needed, or to construct more complex similarity functions [?, ?, ?]. However, in most datasets published on the Web, keys are not available, and even experts may find them difficult to determine.

Key discovery approaches in the Semantic Web context include:

- The approach [?] discovers pseudo keys, which do not follow the OWL2 semantics [?] of a key. These keys are useful when the local completeness of the data is known.
- The approach [?] discovers OWL2 keys in clean data without errors or duplicates, but cannot handle large-scale Web data.

2.2.2 Challenges in Key Discovery

Data published on the Web are often generated automatically and may contain errors or duplicates. These challenges can cause relevant keys to be lost. For example, in a dataset where two employees share the same identifier (ID), ID will not be considered a key. Allowing for some exceptions can cover this issue.

The number of keys discovered in a dataset may be low. Even if a set of properties is not a strict key, it can still generate useful links [?]. For example, a restaurant telephone number is typically a key, but two restaurants in the same location may share a phone number. Despite not being a key, this property is still useful for linking.

The SAKey tool overcomes these challenges.

2.3 SAKey Tool for Key Discovery

2.3.1 Overview

SAKey (Scalable Almost Key Discovery) is an approach to discovering keys in RDF data efficiently [?]. Dynamic detection of data characteristics to reduce search space and can discover keys in datasets with erroneous data or duplicates (almost keys). SAKey follows OWL2 semantics.

Almost Key is a set of properties that is not a key due to a few exceptions (i.e., resources that do not respect the key constraint). The set of almost keys is derived from non-keys found in the data. **Non-Key (n-Non Key)** is a set of properties that fails to be a key due to at least n different resources sharing the same values. A set of properties is an **n-almost key** if at most n instances share the same values for that set of properties.

SAKey can scale on large datasets by applying a number of filtering and pruning techniques that reduce the requirements of time and space.

2.3.2 SAKey Limitation

Despite its scalability and effectiveness, SAKey has some limitations. It only accepts RDF datasets in the N-Triples (NT) format. Therefore, if RDF data is provided in another serialization format—such as Turtle(ttl), RDF/XML/RDFXML, or N-Quads (NQ) must be converted to NT before it can be processed by SAKey.

In this project, we addressed this limitation by developing a conversion step that transforms RDF datasets from various formats into NT format. For example, the OAEI Restaurants1 dataset, which is already in NT format, was used directly, while Open Restaurant Applications dataset were successfully converted from rdf format to N-Triples (NT) format prior to key generation.

2.3.3 SAKey Parameters

- **File Path:** The RDF dataset file where key discovery takes place. In this work we assume this file contains instances of a single class.
- **Number of Exceptions (n):** Defines how many exceptions are allowed in discovering almost keys.

In this work we considered the number of exceptions n=1, so the output will be the following two sets of keys:

```
n-non keys is 1-non keys
n-1 almost keys is 0-almost keys
```

2.3.4 Running SAKey

SAKey is executed using the following command:

```
java -jar "jar_fullpath"/SAKey.jar "file_fullpath" n
```

2.3.5 Generating keys process

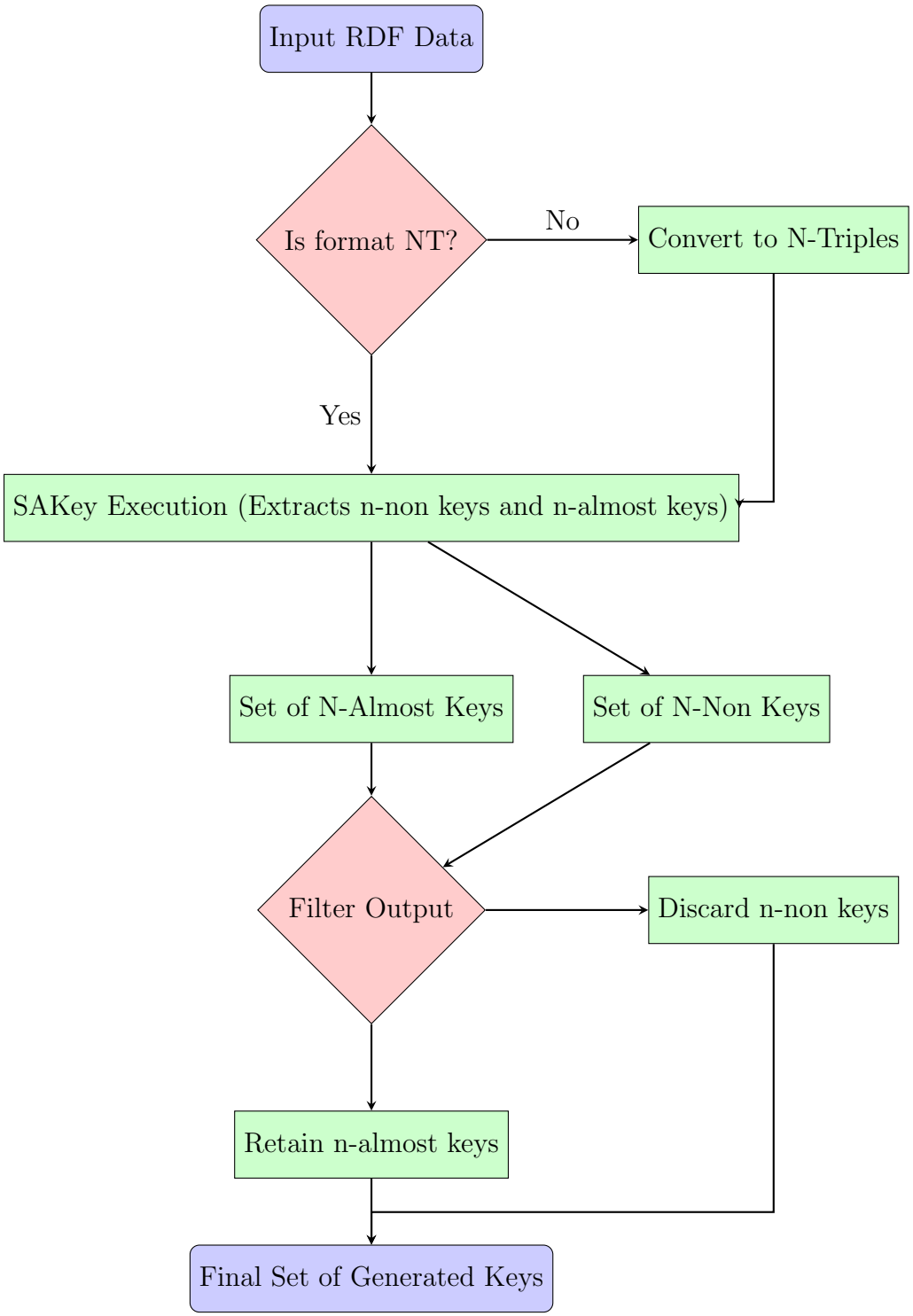


Figure 2.1: Key Generation Process with Format Conversion

In this project, we consider n=1, since **0-almost keys** provide strict uniqueness constraints. Therefore, they are used for SHACL shape generation in this work.

2.4 Generating SHACL Shapes

2.4.1 Overview

With the rapid expansion of heterogeneous RDF data graphs on the Web, ensuring data quality and consistency has become a big challenge. RDF graphs, represented using the RDF standard and queried via SPARQL[1], often contain inconsistencies due to errors and incompleteness. To address this, the W3C introduced **SHACL (Shapes Constraint Language)** as a standard for defining and validating constraints on RDF data.

SHACL shapes define structural and integrity constraints on RDF data, allowing for the identification of **non-conforming** data instances. These constraints are expressed as **SHACL NodeShapes**, which specify rules that RDF nodes must satisfy. The validation process involves evaluating a **SHACL shapes graph** G_s against an **RDF data graph** G_d to detect violations of constraints and rules.

Several approaches exist for extracting SHACL constraints from RDF data, including:

- **Ontology-based approaches** [**?, ?**]: Extracting SHACL shapes from ontologies (schemas).
- **Data-driven mining approaches** [**?, ?, ?**]: extracting SHACL shapes by mining regularities from RDF facts.
- **Hybrid approaches** [**?**]: Combining ontology-based and data-driven approaches.
- **Discovering SHACL constraints based on gramatical evolution** [**?**]: generating candidate SHACL shapes using a BNF grammar and RDF data as input.

Our approach consists of two main **scenarios** for generating SHACL shapes:

1. Using discovered keys (K) to generate SHACL shapes across RDF data sources with the same property structure.
2. Using discovered keys (K) and translated properties (P) to generate SHACL shapes for RDF data sources that share similar structures but have different property names.

[1]https://www.w3.org/TR/rdf-sparql-query/

2.4.2 Generating SHACL Shapes Based on Keys

Given multiple RDF datasets $D_1, D_2, ..., D_n$ that share a common property structure, we aim to generate **SHACL shapes** based on discovered keys without re-extracting them for each dataset. For example, if we have a dataset for children's books and another dataset for adult books, to generate SHACL shapes for the datasets, we first generate keys from the children's or adult books dataset. Then, we use these keys to generate SHACL shapes, in this way we do key discovery and generating SHACL shapes one time for multiple datasets that have the same property structure, then we can do the validation process to determine which keys remain effective in the other similar datasets.

Let K be the set of discovered 0-almost keys from a dataset D_1, which means it enforces uniqueness.

For any dataset D_j $(j > 1)$, we generate **SHACL shapes** based on the 0-almost keys that are generated for D_1 while **0-almost keys** are used to construct constraints of the form:

$$\forall k_i \in K, \quad k_i = \{p_1, p_2, ..., p_m\} \Rightarrow \text{SHACL Shape}$$

Each key k_i defines an RDF node shape with constraints ensuring the uniqueness of properties $p_1, p_2, ..., p_m$, expressed in SHACL. So suppose $k_1 = \{p_1, p_2\} \in K$, the NodeShape for p_1, p_2 expressed as:

```
ex:Shape01  a          sh:NodeShape;
        sh:property    [ sh:datatype  xsd:string;
                         sh:minCount  "1"^^xsd:int;
                         sh:maxCount  "1"^^xsd:int;
                         sh:path      ex:p1
                       ];
        sh:property    [ sh:datatype  xsd:string;
                         sh:minCount  "1"^^xsd:int;
                         sh:maxCount  "1"^^xsd:int;
                         sh:path      ex:p2
                       ];
```

Figure 2.2: Generating SHACL NodeShape based on keys

This ensures that for all datasets D_j, the same **SHACL shapes** apply, allowing the check of key effectivity without repeated key discovery or the generation of SHACL shapes.

2.4.3 Generating SHACL Shapes Based on Keys and Translated Properties

In some cases, RDF datasets have similar property structures but use **different property names**. Consider two book datasets:

- B_1 with properties $\{\text{hasAuthor}, \text{hasTitle}\}$
- B_2 with properties $\{\text{author}, \text{title}\}$

Let's suppose $k_1 \in K$, where K is the set of generated keys for B_1, and $k_1 = \{\text{hasAuthor}, \text{hasTitle}\}$. We cannot use these keys alone to generate SHACL shapes for B_2. Instead, we need to apply them to B_2 using **property translation** rather than rediscovering keys.

Thus, before generating SHACL shapes, we must define the property mappings:

$$hasAuthor \equiv author$$

$$hasTitle \equiv title$$

Given multiple datasets $D_1, \ldots, D_m$, where each D_j $(j > 1)$ is an RDF dataset that has the same property structure as D_1 but with different property names, we define:

- K as the set of generated 0-almost keys for D_1.

- For any $k_i \in K$, let:

$$k_i = \{p_1, p_2, \ldots, p_m\}$$

where $P = \{p_1, p_2, \ldots, p_m\}$ is the set of properties in D_1.

- Let P'_j be the set of translated properties in D_j.

Since $P \neq P'_j = \{p'_{j1}, p'_{j2}, \ldots, p'_{jm}\}$, we cannot directly use K to generate SHACL shapes for D_j. Thus, before generating SHACL shapes, we first define property equivalences as:

$$p'_{j1} \equiv p_1, \quad p'_{j2} \equiv p_2, \quad \ldots, \quad p'_{jm} \equiv p_m$$

After establishing these equivalences, we use the translated properties with the keys from K to generate SHACL shapes for D_j.

$$\forall k_i \in K, \quad k_i = \{p_1, p_2, ..., p_m\}, p'_{j1} \equiv p_1, \quad p'_{j2} \equiv p_2, \quad \ldots, \quad p'_{jm} \equiv p_m \Rightarrow \text{SHACL Shape}$$

For example:

Let $k_1 \in K$ be an **0-almost key** in D_1 composed of properties p_1, p_2:

$$k_1 = \{p_1, p_2\}$$

This can be used to generate the SHACL shapes same as in 2.2.

For another dataset D_2, has properties p'_1 and p'_2, we can define **equivalent translated properties**:

$$p'_{21} \equiv p_1$$

$$p'_{22} \equiv p_2$$

Thus, the corresponding **SHACL shape for D_2 is**:

```
ex:Shape01  a          sh:NodeShape;
        sh:property    [ sh:datatype  xsd:string;
                         sh:minCount  "1"^^xsd:int;
                         sh:maxCount  "1"^^xsd:int;
                         sh:path      ex:p1'
                       ];
        sh:property    [ sh:datatype  xsd:string;
                         sh:minCount  "1"^^xsd:int;
                         sh:maxCount  "1"^^xsd:int;
                         sh:path      ex:p2'
                       ];
```

Figure 2.3: Generate SHACL Nodeshapes based on keys and translated properties

This method allows to check which **SHACL constraints remain valid across RDF datasets** with different but **structurally equivalent properties**, avoiding redundancy in key generation.

After generating SHACL Nodeshapes, two SHACL constraints will be created for each key, whether using property names or the translated ones.

2.4.4 Uniqueness Constraint

This constraint ensures that keys maintain their uniqueness across similar RDF datasets.

Given a K set of *0-almost keys* extracted from an RDF dataset D_1, we know that each $k_i \in K$ is unique in D_1. However, when applying these keys to another RDF dataset D_2 with the same structure, we must validate whether k_i remains unique.

- Let D_1 and D_2 be two RDF datasets with a similar structure.
- Let e_1 and e_2 be two instances of data in D_1.
- Let k_1 be an *0-almost key* composed of properties p_1, p_2:

$$k_1 = \{p_1, p_2\}$$

- We assume k_1 is unique in D_1, meaning:

$$\forall e_1, e_2 \in D_1, \quad e_1 \neq e_2 \Rightarrow (p_1(e_1), p_2(e_1)) \neq (p_1(e_2), p_2(e_2))$$

- We now check whether k_1 remains unique in D_2, ensuring that for every entity $e \in D_2$, there is at most one occurrence of the combination (p_1, p_2):

$$\forall e'_1, e'_2 \in D_2, \quad e'_1 \neq e'_2 \Rightarrow (p_1(e'_1), p_2(e'_1)) \neq (p_1(e'_2), p_2(e'_2))$$

  Or if SHACL shapes are generated based on keys and translated properties in D_2:

$$p'_{21} \equiv p_1$$

$$p'_{22} \equiv p_2$$

  then we need to check:

$$\forall e'_1, e'_2 \in D_2, \quad e'_1 \neq e'_2 \Rightarrow (p'_2 1(e'_1), p'_2 2(e'_1)) \neq (p'_2 1(e'_2), p'_2 2(e'_2))$$

This SHACL constraint is generated using SPARQL Query. **SPARQL** (SPARQL Protocol and RDF Query Language) is an RDF query language that can be used to express queries across diverse data sources, whether the data is stored natively as RDF or viewed as RDF via middleware.

SPARQL Query: The following SHACL constraint detects duplicate values for p_1, p_2 in D_2, without enforcing an explicit equivalence (i.e., without using `owl:sameAs`). It finds entities e_1 and e_2 that have the same values for p_1 and p_2, ensuring $e_1 \neq e_2$:

```
sh:sparql    [ a           sh:SPARQLConstraint;
               sh:message  "Duplicate detected for properties: p1, p2";
               sh:select   "PREFIX ex: <http://example.com/ns#>
                           SELECT ?this
                           WHERE {
                               ?this ex:p1 ?p1 .
                               ?other ex:p1 ?p1 .
                               ?this ex:p2 ?p2 .
                               ?other ex:p2 ?p2 .
                               FILTER (?this != ?other)
                           }"
             ];
```

Figure 2.4: Generating uniqueness constraint

If duplicates exist $(p_1(e_1), p_2(e_1)) = (p_1(e_2), p_2(e_2))$, the message: *"Duplicate detected for properties: p1, p2"* will be triggered.

2.4.5 SameAs Rule

The second constraint is the **SameAs Rule**, which ensures that duplicate entities are explicitly linked using `owl:sameAs`.

owl:sameAs is the built-in OWL property links an individual to an individual. Such an owl:sameAs statement indicates that two URI references actually refer to the same thing: the individuals have the same "identity".

For individuals such as "people" this notion is relatively easy to understand. For example, we could state that the following two URI references actually refer to the same person:

```
<rdf:Description rdf:about="#William_Jefferson_Clinton">
  <owl:sameAs rdf:resource="#BillClinton"/>
</rdf:Description>
```

The **owl:sameAs** statements are often used in defining mappings between ontologies. This sameAs rule is generated using a SHACL rule defined via a SPARQL rule.
If two entities (e'_1,e'_2) in D_2 have the same values for properties $p'_2 1, p'_2 2$, they should be linked by `owl:sameAs`. This constraint ensures that duplicate entities are formally recognized as equivalent.

SHACL rule: The following rule constructs an `owl:sameAs` triple for entities with identical p_1, p_2:

```
sh:rule      [ a           sh:TripleRule;
               sh:select   "PREFIX ex: <http://example.com/ns#>
                           PREFIX owl: <http://www.w3.org/2002/07/owl#>
                           CONSTRUCT {
                               ?this owl:sameAs ?other .
                           }
                           WHERE {
                               ?this ex:p1 ?p1 .
                               ?other ex:p1 ?p1 .
                               ?this ex:p2 ?p2 .
                               ?other ex:p2 ?p2 .
                               FILTER (?this != ?other)
                           }"
             ];
```

Figure 2.5: Generating sameAs rule

SPARQL Constraint: To verify that duplicates are correctly marked as `owl:sameAs`, we define the following SPARQL query:

```
sh:sparql    [ a          sh:SPARQLConstraint;
               sh:message "Duplicate with the same properties (p1, p2)
               must be marked as owl:sameAs.";
               sh:select  "PREFIX ex: <http://example.com/ns#>
                          PREFIX owl: <http://www.w3.org/2002/07/owl#>
                          SELECT ?this ?other
                          WHERE {
                            ?this ex:p1 ?p1 .
                            ?other ex:p1 ?p1 .
                            ?this ex:p2 ?p2 .
                            ?other ex:p2 ?p2 .
                            FILTER (?this != ?other)
                            FILTER NOT EXISTS {?this owl:sameAs ?other}
                           }"
             ];
```

Figure 2.6: Generating sameAs SHACL constraint

If the constraint is violated ($(p_1(e_1), p_2(e_1)) = (p_1(e_2), p_2(e_2))$ and e_1, e_2 are not linked with *owl:sameAs*), the message: *"Duplicate with the same properties must be marked as owl:sameAs."* will be triggered.

2.4.6 Defining the Target Nodes

To apply the rule only to relevant entities, we define a target constraint:

```
sh:target [ a sh:SPARQLTarget;
            sh:select "PREFIX ex: <http://example.com/ns#>
                       SELECT ?this
                       WHERE {
                           ?this ex:p1 ?p1 .
                           ?other ex:p1 ?p1 .
                           ?this ex:p2 ?p2 .
                           ?other ex:p2 ?p2 .
                           FILTER (?this != ?other)
                       }"
          ];
```

Figure 2.7: Generate target constraint

This query ensures that only entities- that has properties p_1 and p_2 - with at least one duplicate are processed.

2.5 Validation Process

2.5.1 Overview:

SHACL validation takes a *data graph* and a *shapes graph* as input, producing a validation report that contains the portions of the source shape responsible for the violation. As the SHACL shapes graph stores all shapes, we can obtain the entire violated shape when a transaction fails. **Validation process** is conformance checking determines whether a dataset satisfies the constraints.

2.5.2 Validation Steps:

1. Convert RDF data format to NT format.
2. Process RDF data to avoid incorrect violations.
3. Apply SHACL constraints to detect duplicates, breached rules and constraints to detect ineffective keys.
4. Validate RDF triples and generate a detailed validation report.

2.5.3 Data Processing Algorithm:

Algorithm 2: RDF processing algorithm

Input: RDF dataset
Output: processed RDF dataset
for each line in RDF file **do**
    Remove empty lines
    Split line into $\langle subject, predicate, object \rangle$
    Remove angle brackets $\langle\rangle$
    Convert predicates to lowercase
    Fix formatting issues
end for
Store processed data

Severity Levels: Each SHACL constraint should define a severity level:

```
sh:severity sh:Violation;
```

This marks the rule as a **violation**, meaning non-compliance should be treated as an error.

2.5.4 Validation Report

The validation report is customized to provide the following:

- Data instances that violate constraints.
- The keys involved in each violation.
- A detailed error message for each issue.
- Number of violations for each constraint.
- The total number of violations.

Let D_3 dataset that has data instances (e_1, e_2), with properties (p_1, p_2), and the pre-generated key $k_1 = \{p_1, p_2\} \in K$:

```
<e1> <p1> <p1(e1)> .
<e1> <p2> <p2(e1)> .
<e2> <p1> <p1(e1)> .
<e2> <p2> <p2(e1)> .
<e2> <p2> <p2(e2)> .
```

The following validation report will be generated as a result of the validation process:

```
Validation Report
Date: 2025-04-23 08:21:27
Findings: 9
Result: Failed
Details:
Message: Duplicate detected for properties: (p1, p2)
Subjects: http://example.com/ns#e1,
http://example.com/ns#p2
Occurrences: 2
- Message: Duplicate with the same properties: (p1, p2)
must be marked as owl:sameAs.
Subjects: http://example.com/ns#e1,
http://example.com/ns#e2
Occurrences: 2
- Message: maxCount[1]: Invalid cardinality: expected max 1: Got count = 2
Subjects: http://example.com/ns#e2
Occurrences: 1
```

2.6 Conclusion

In this chapter, we outline a systematic approach for generating SHACL shapes and validating them against RDF data sources. The process was divided into four key steps: converting to NT format, generating keys, constructing SHACL shapes, and validating these shapes against RDF data. The first step was checking the RDF data format, and converting any RDF data format to N-Triples (NT) format, to be able to use SAKEY tool in the next step. The second step involved generating keys using the SAKey tool that accepts only N-Triples (NT) RDF data format, which efficiently identifies *0-almost keys* in the RDF dataset. Using these almost keys, we were able to generate SHACL shapes.

The third step was how SHACL shapes are created from discovered keys, ensuring key uniqueness between data instances, the sameAs link duplicated data instances, and structural integrity across RDF datasets that share the same property structure. In cases where the datasets shared similar structures but used different property names, translated properties were used to generate SHACL shapes to NT fornat.

Finally, process data and validate these generated SHACL constraints against RDF data to violate breached constraints with messages that explain the violation, and generate validation report with details about violations, number of violations per breached constraint, detect data instances that breached the constraints, and the total number of violations.

In general, this approach simplifies the process of generating and validating SHACL shapes, improving the consistency of RDF datasets.

List of Figures

2.1 Key Generation Process with Format Conversion 9
2.2 Generating SHACL NodeShape based on keys 11
2.3 Generate SHACL Nodeshapes based on keys and translated properties . . . 13
2.4 Generating uniqueness constraint 15
2.5 Generating sameAs rule 16
2.6 Generating sameAs SHACL constraint 17
2.7 Generate target constraint 18

List of Algorithms

1 RDF File Conversion Process 6
2 RDF processing algorithm 19

List of Tables